TRABALHO PRÁTICO
DIVISÃO E CONQUISTA

Implementação e comparação experimental de algoritmos em JavaScript (Node.js)

Integrantes	[Nome do integrante 1] e [Nome do integrante 2]
Disciplina	[Nome da disciplina]
Linguagem	JavaScript com Node.js
Repositório	github.com/gmenesesalc/div-conquer
Data	21 de outubro de 2026

Resumo. Este trabalho implementa e compara força bruta e divisão e conquista para dois problemas: subarranjo de soma máxima e par de pontos mais próximos. A avaliação combina análise assintótica, testes automatizados e medições experimentais em entradas crescentes. Os dados disponíveis indicam melhor escalabilidade das soluções de divisão e conquista. As medições apresentadas foram obtidas em ambiente Linux virtualizado e devem ser repetidas no computador da dupla antes da entrega definitiva.

Palavras-chave: algoritmos; força bruta; divisão e conquista; complexidade; análise experimental.

1. Objetivo e metodologia

O objetivo é implementar soluções corretas para os dois problemas e comparar a abordagem direta com a técnica de divisão e conquista. A análise considera complexidade temporal e espacial, testes de correção e comportamento do tempo de execução à medida que o tamanho da entrada cresce.

Os benchmarks registrados utilizaram Node.js v22.16.0 em Linux x64 virtualizado, com processador reportado como AMD EPYC 9V74 e cinco CPUs lógicas disponíveis ao ambiente. Foram usados os tamanhos n = 100, 200, 400, 800, 1600, 3200 e 6400, com cinco amostras por algoritmo e tamanho. As funções foram aquecidas antes das medições e chamadas rápidas foram medidas em pequenos lotes para reduzir o ruído do cronômetro. Os valores são médias por chamada, em milissegundos.

As entradas de benchmark são geradas deterministicamente. Antes da cronometragem, as implementações comparam suas respostas para verificar consistência. Os tempos dependem da máquina, da versão do Node.js e da carga do sistema; portanto, estes números representam uma execução de referência, não uma medição do computador pessoal da dupla.

2. Etapa 1 — Subarranjo de soma máxima

Dado um vetor não vazio, o objetivo é encontrar um intervalo contíguo cuja soma seja máxima. Os índices são baseados em zero e o índice final é inclusivo. Em caso de empate, escolhe-se o intervalo com menor índice inicial e, depois, menor índice final. Se todos os elementos forem negativos, a resposta é o maior elemento do vetor.

2.1 Força bruta

A solução enumera todos os pares de índices inicial e final e mantém uma soma acumulada ao avançar o fim do intervalo. Como existem $n(n + 1)/2$ intervalos, o tempo é $\Theta(n^2)$ e o espaço auxiliar é $\Theta(1)$.

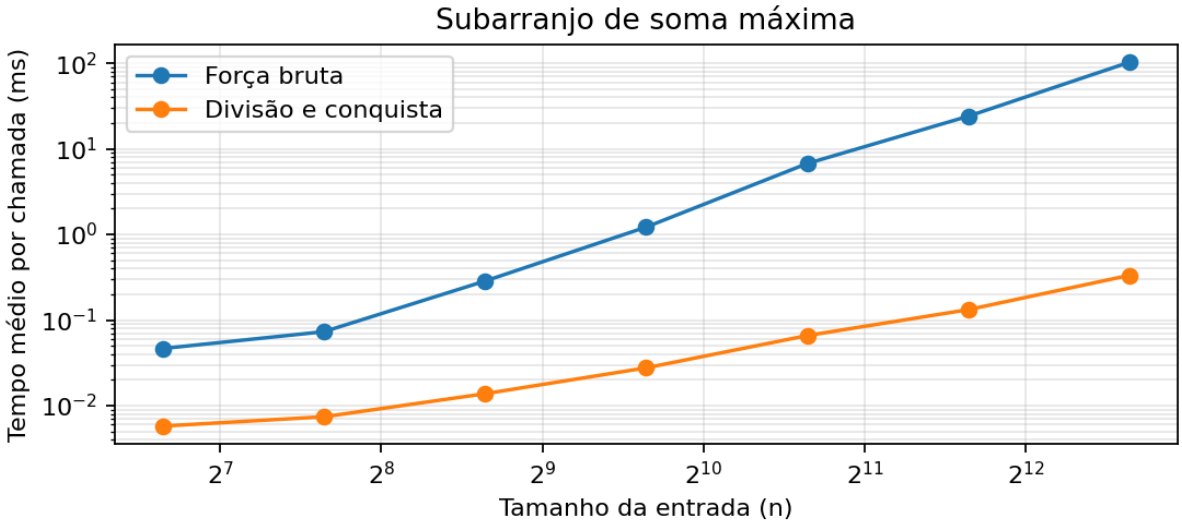
2.2 Divisão e conquista

O vetor é dividido em duas metades. O melhor intervalo pode estar inteiramente à esquerda, inteiramente à direita ou cruzar o meio. Para o caso cruzado, calcula-se em tempo linear o melhor sufixo da metade esquerda e o melhor prefixo da metade direita. A recorrência é $T(n) = 2T(n/2) + \Theta(n)$; pelo Teorema Mestre, o tempo é $\Theta(n \log n)$, com $\Theta(\log n)$ de pilha recursiva.

2.3 Resultados experimentais — soma máxima

A tabela apresenta o tempo médio por chamada na execução de referência.

n	Força bruta (ms)	Divisão e conquista (ms)
100	0.046453	0.005760
200	0.073037	0.007393
400	0.285409	0.013757
800	1.219414	0.027591
1600	6.743752	0.065790
3200	24.099554	0.132163
6400	103.063674	0.332856



A força bruta cresce aproximadamente de forma quadrática, pois avalia todos os intervalos possíveis. A divisão e conquista cresce mais lentamente, conforme esperado para Θ(n log n). Em n = 6400, os tempos registrados foram 103,063674 ms para força bruta e 0,332856 ms para divisão e conquista. Pequenas oscilações são normais; a análise considera a tendência em vários tamanhos.

3. Etapa 2 — Par de pontos mais próximos

Dado um conjunto de pontos no plano, o objetivo é encontrar o par cuja distância euclidiana é mínima. Os índices retornados são baseados em zero e correspondem à ordem original dos pontos. A entrada deve conter pelo menos dois pontos com coordenadas finitas. Pontos duplicados são permitidos e podem resultar em distância zero.

3.1 Força bruta

A solução compara todos os pares distintos. Para n pontos existem $n(n-1)/2$ pares, resultando em $\Theta(n^2)$ tempo. A implementação compara quadrados de distâncias para evitar calcular raízes quadradas em cada comparação.

3.2 Divisão e conquista

Os pontos são ordenados inicialmente por x e por y. A lista ordenada por x é dividida em duas metades e cada parte é resolvida recursivamente. A menor distância encontrada determina δ. Na combinação, examina-se a faixa central com distância horizontal de até δ da linha divisória; os pontos já estão ordenados por y, e cada ponto é comparado com no máximo os sete seguintes. O argumento geométrico de empacotamento limita as comparações necessárias. Subproblemas com até três pontos são resolvidos por força bruta em tempo constante.

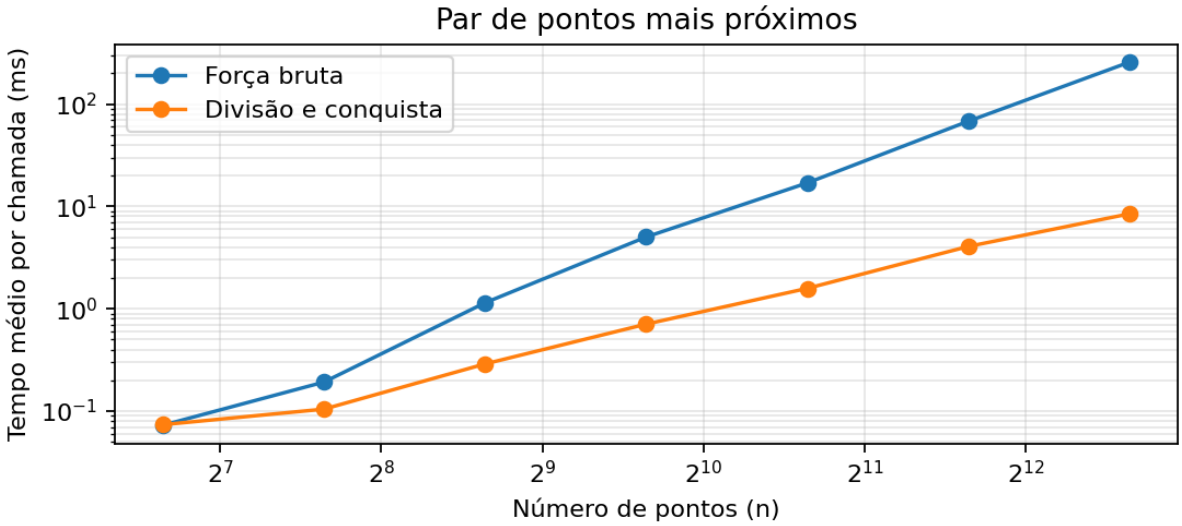
A ordenação inicial custa $\Theta(n \log n)$ e a recorrência da fase recursiva é $T(n) = 2T(n/2) + \Theta(n)$, resultando em $\Theta(n \log n)$ de tempo total. Nesta implementação, as partições e listas auxiliares são recriadas nos níveis recursivos; o espaço auxiliar pode chegar a $\Theta(n \log n)$.

Critério de empate. A implementação escolhe o par de índices lexicograficamente menor entre pares com a mesma distância. Os testes verificam a distância mínima e se os índices retornados identificam um par que atinge essa distância.

3.3 Resultados experimentais — pontos mais próximos

Tempos médios por chamada na execução de referência.

n	Força bruta (ms)	Divisão e conquista (ms)
100	0.072812	0.073701
200	0.192241	0.104598
400	1.144405	0.289126
800	5.028482	0.707849
1600	16.958648	1.582213
3200	67.987173	4.064112
6400	256.680733	8.467707



A força bruta cresce rapidamente porque precisa avaliar todos os pares. A divisão e conquista tem custos de ordenação e recursão que podem torná-la semelhante ou ligeiramente mais lenta em entradas pequenas, mas escala melhor para entradas maiores. Em n = 6400, foram registrados 256,680733 ms para força bruta e 8,467707 ms para divisão e conquista.

4. Testes e validação

A suíte automatizada registrada no projeto contém 42 testes aprovados na execução anterior. Ela cobre casos de borda e compara as implementações. Para soma máxima, os testes incluem vetor unitário, números negativos, zeros, valores positivos, empates e intervalos que cruzam o meio. Para pontos mais próximos, incluem pares horizontais, verticais e diagonais, pontos colineares, duplicatas, empates e pares que cruzam a divisão. Também são verificadas entradas inválidas e comparações com força bruta em entradas determinísticas.

Os testes podem ser executados com npm test. Para repetir as medições, use npm run benchmark. Para regenerar os gráficos, o projeto fornece python3 tools/plot-results.py, que requer Python 3 e matplotlib. Após repetir os experimentos, os CSVs, a configuração do ambiente e os gráficos devem ser atualizados juntos.

5. Comparação das abordagens

Problema	Força bruta	Divisão e conquista	Diferença principal
Soma máxima	Θ(n²) tempo Θ(1) espaço	Θ(n log n) tempo Θ(log n) pilha	Evita enumerar todos os intervalos.
Pontos mais próximos	Θ(n²) tempo	Θ(n log n) tempo espaço auxiliar até Θ(n log n)	Reduz as comparações usando a faixa central.

6. Conclusão

A análise teórica e as medições disponíveis são consistentes com a vantagem assintótica da divisão e conquista nos dois problemas. As soluções de força bruta, com tempo Θ(n²), são diretas e servem como referência para validar a correção. As soluções de divisão e conquista, com tempo Θ(n log n), tornam-se mais eficientes à medida que a entrada cresce. No problema dos pontos mais próximos, a ordenação inicial e a análise limitada da faixa central evitam comparar todos os pares.

Antes da entrega, a dupla deve preencher os nomes e a disciplina, repetir testes e benchmarks no computador que será descrito, atualizar os CSVs e gráficos e conferir o limite de oito páginas. Os números deste documento correspondem à execução Linux virtualizada já registrada no repositório.

Repositório: https://github.com/gmenesesalc/div-conquer